Spotify Genre Classification Using a Deep Learning MLP

AI 100 - Midterm Project

Marco King

February 2026

1. Problem Definition and Dataset Curation

Problem

Music streaming platforms like Spotify organize millions of tracks by genre, but genre labels are often applied manually or inconsistently. This project explores whether a deep learning model can automatically classify songs into broad genre categories using only their numeric audio characteristics - features like danceability, energy, tempo, and loudness - without listening to the actual audio.

This is framed as a multi-class classification problem: given 13 audio features for a track, predict which of 10 genre categories it belongs to.

Dataset

The dataset is the Spotify Tracks Dataset from Kaggle, containing approximately 114,000 tracks. Each track includes 13 numeric audio features extracted by Spotify's API:

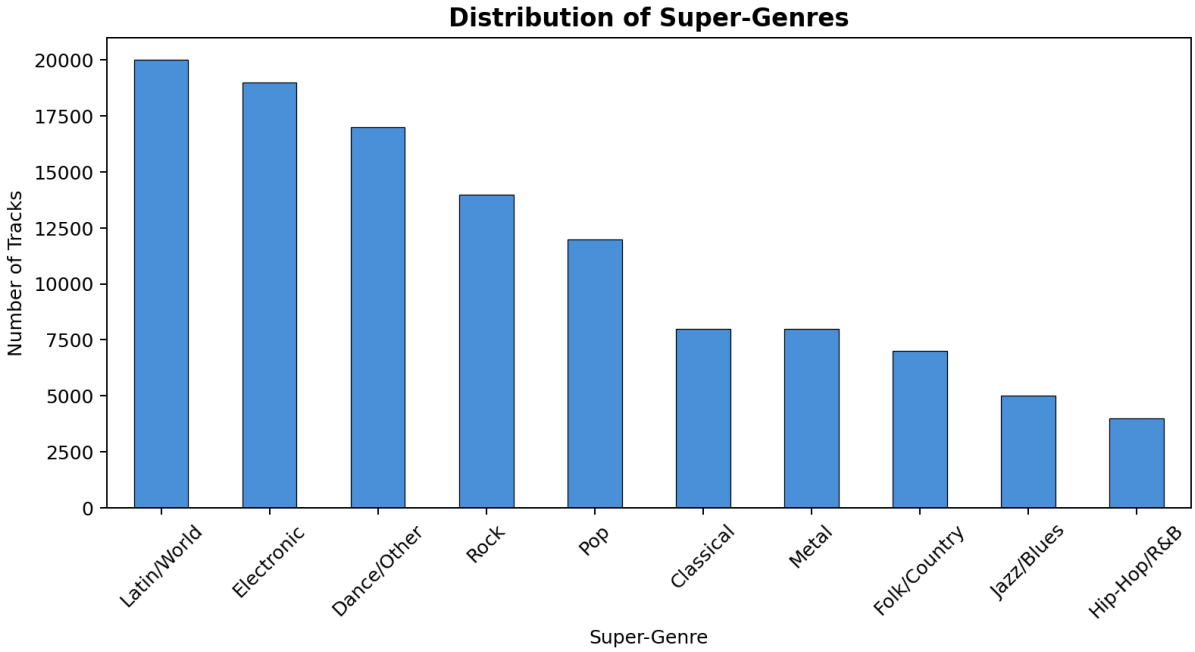
Feature	Description	Range
Danceability	How suitable for dancing	0.0 - 1.0
Energy	Intensity and activity	0.0 - 1.0
Key	Musical key	0 - 11
Loudness	Overall loudness (dB)	-60 - 0
Mode	Major (1) or minor (0)	0 or 1
Speechiness	Spoken words presence	0.0 - 1.0
Acousticness	Acoustic confidence	0.0 - 1.0
Instrumentalness	No vocals prediction	0.0 - 1.0
Liveness	Live recording probability	0.0 - 1.0
Valence	Musical positivity	0.0 - 1.0
Tempo	Tempo in BPM	~0 - 250
Time Signature	Est. time signature	1 - 5
Duration (ms)	Track length	varies

Genre Grouping

The original dataset contains 114 fine-grained genre labels. Many are closely related and have too few samples individually for reliable classification. To create a tractable problem, the 114 genres were grouped into 10 super-genres:

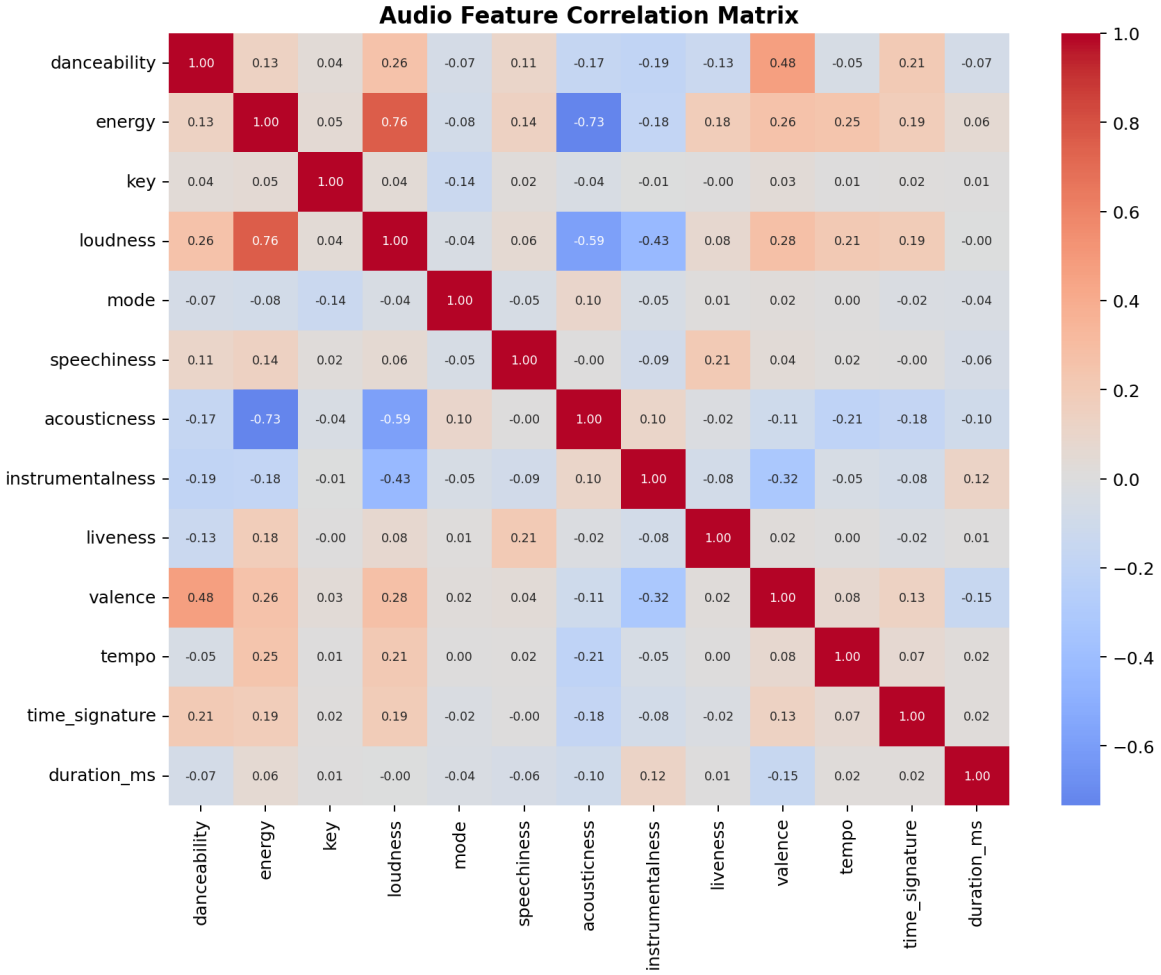
Super-Genre	Example Sub-Genres	Tracks
Rock	rock, alt-rock, alternative, hard-rock (+10)	14,000
Metal	metal, heavy-metal, death-metal, black-metal (+4)	8,000
Pop	pop, indie-pop, synth-pop, k-pop (+8)	12,000
Electronic	edm, electro, electronic, house (+15)	19,000
Hip-Hop/R&B	hip-hop, r-n-b, reggaeton, dancehall	4,000
Jazz/Blues	jazz, blues, soul, funk (+1)	5,000

Classical	classical, piano, opera, guitar (+4)	8,000
Folk/Country	folk, country, acoustic, bluegrass (+3)	7,000
Latin/World	latin, latino, salsa, samba (+16)	20,000
Dance/Other	dance, club, disco, party (+13)	17,000

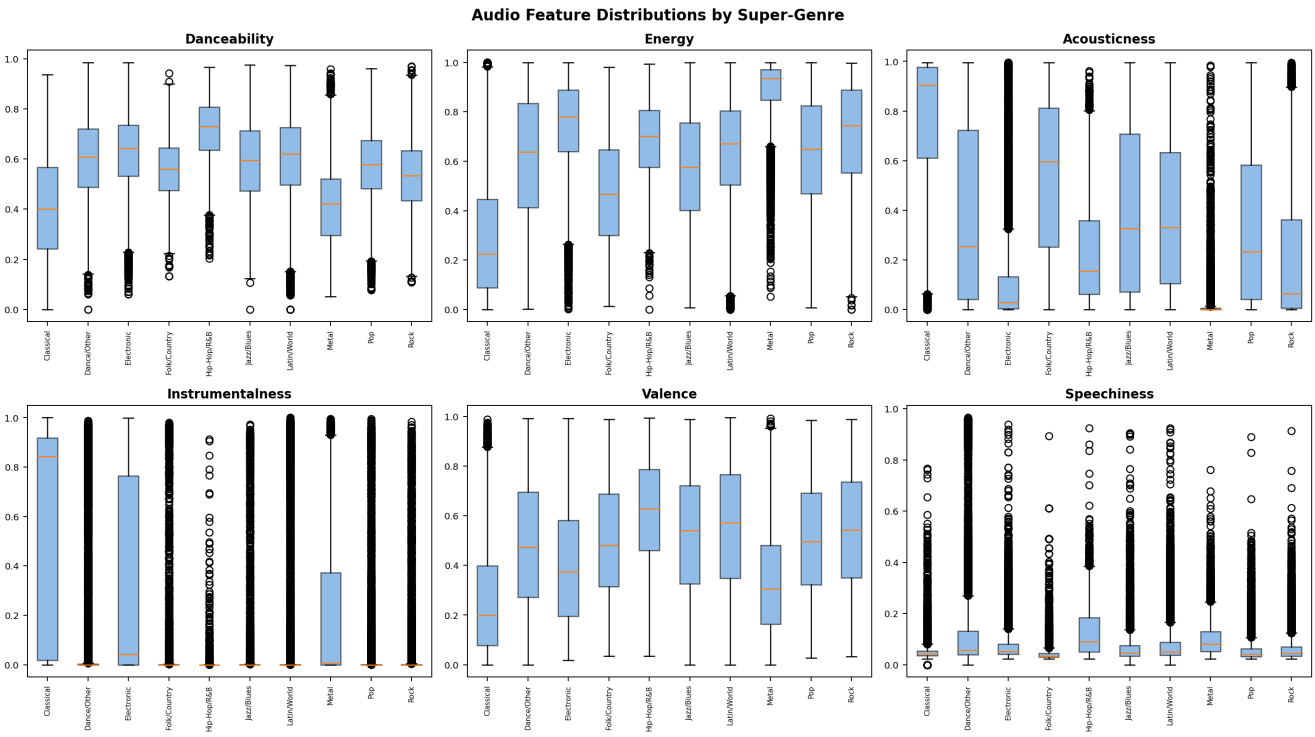


The data was split 80/20 into training (91,200 samples) and test (22,800 samples) sets using stratified sampling to preserve class proportions. All features were standardized (zero mean, unit variance) using the training set statistics only.

Feature Analysis



The correlation matrix shows most features are relatively independent, with the notable exception of energy and loudness (r=0.76). This low multicollinearity is favorable for classification, as each feature contributes relatively unique information.



The box plots reveal distinct feature signatures across genres. Classical music stands out with high acousticness and

instrumentalness. Metal shows high energy. Hip-Hop/R&B has elevated speechiness. These patterns suggest that audio features carry real discriminative signal.

2. Deep Learning Model

Baseline: Logistic Regression

Before building a deep learning model, a logistic regression classifier was trained as a baseline to establish a performance floor. Logistic regression is a linear model - it can only learn linear decision boundaries between classes. This provides a reference point to measure how much the neural network improves by learning non-linear patterns.

Model Architecture: Multi-Layer Perceptron (MLP)

The primary model is a Multi-Layer Perceptron (MLP) implemented in PyTorch - a feedforward neural network of fully connected layers. It is well-suited for tabular data because it can learn complex non-linear relationships between inputs and outputs.

```
Input (13 features)
  |
  +-- Linear(13 -> 256) -> BatchNorm -> ReLU -> Dropout(0.3)
  |
  +-- Linear(256 -> 128) -> BatchNorm -> ReLU -> Dropout(0.3)
  |
  +-- Linear(128 -> 64)  -> BatchNorm -> ReLU -> Dropout(0.2)
  |
  +-- Linear(64 -> 10)   -> Output (10 super-genres)

Total trainable parameters: ~46,000
```

Key design choices:

- **Batch Normalization:** Normalizes the input to each layer, stabilizing training and allowing higher learning rates.
- **ReLU Activation:** Introduces non-linearity that allows the network to learn patterns logistic regression cannot.
- **Dropout (30% and 20%):** Randomly zeroes out neurons during training to prevent overfitting. Higher dropout (30%) in earlier, wider layers.

Training Configuration

Parameter	Value
Loss Function	CrossEntropyLoss
Optimizer	Adam (lr=0.001, weight_decay=1e-4)
LR Scheduler	ReduceLROnPlateau (patience=5, factor=0.5)
Batch Size	256
Epochs	50
Feature Scaling	StandardScaler (fit on train only)

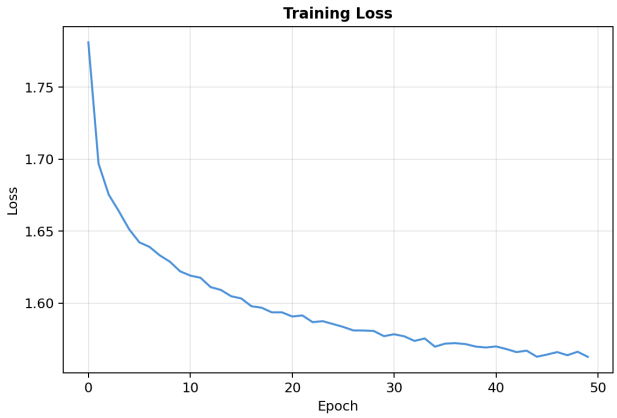
3. Results

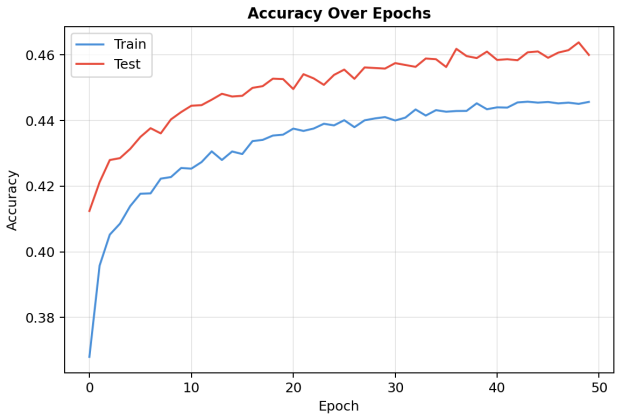
Model Comparison

Model	Test Accuracy
Logistic Regression (baseline)	38.1%
MLP Neural Network	46.0%
Relative Improvement	+20.6%

For context, random guessing among 10 classes would yield ~10% accuracy. Both models significantly outperform chance. The MLP's ~20% relative improvement over logistic regression demonstrates that there are meaningful non-linear patterns in audio features that a deep learning model can capture but a linear model cannot.

Training Dynamics





Training loss decreased steadily over 50 epochs, while test accuracy plateaued around epoch 30-40, indicating convergence. The small gap between training and test accuracy suggests that dropout and batch normalization effectively prevented severe overfitting.

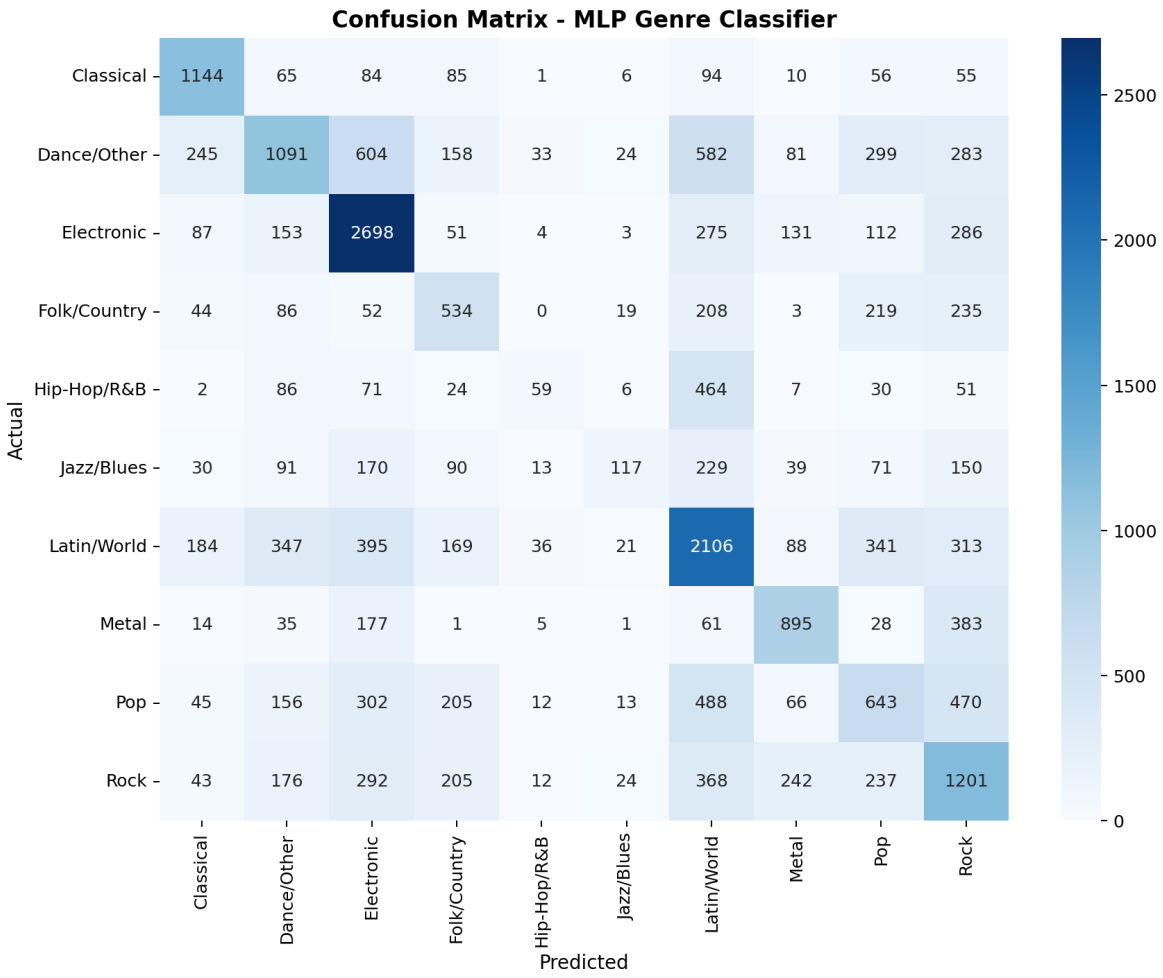
Per-Genre Performance

Super-Genre	Precision	Recall	F1-Score	Support
Classical	0.62	0.71	0.67	1600
Dance/Other	0.48	0.32	0.38	3400
Electronic	0.56	0.71	0.62	3800
Folk/Country	0.35	0.38	0.37	1400
Hip-Hop/R&B	0.34	0.07	0.12	800
Jazz/Blues	0.50	0.12	0.19	1000
Latin/World	0.43	0.53	0.47	4000
Metal	0.57	0.56	0.57	1600
Pop	0.32	0.27	0.29	2400
Rock	0.35	0.43	0.39	2800

Best-classified genres: Classical, Electronic, and Metal. These have distinct audio signatures - classical music is high in acousticness, metal is high in energy and loudness, and electronic music has distinctive danceability and tempo patterns.

Hardest genres: Hip-Hop/R&B and Jazz/Blues. These had the fewest training samples and share audio characteristics with other genres. Hip-hop's danceability and energy overlap heavily with Pop and Dance.

Confusion Matrix



The confusion matrix reveals that most misclassifications occur between genres with similar audio profiles - Rock is often confused with Metal and Pop; Dance/Other is spread across multiple categories, reflecting its catch-all nature.

4. Lessons and Experience

How This Project Was Built

As the assignment permits, I used an LLM (Claude) to help write the code for this project. My role was defining the problem, choosing the dataset, making design decisions (like how to group the 114 genres into 10 super-genres), and making sure I understood what the code was actually doing at each step. The LLM handled the implementation details - writing the PyTorch model class, setting up the training loop, and generating visualizations. I then reviewed the code, ran it locally, and interpreted the results.

I think this workflow is worth reflecting on honestly, because the hardest part of this project was not getting the code to run - it was making sure I actually understood what was happening. It would have been easy to let the LLM do everything without asking questions, but I made a point of understanding each decision: why we used CrossEntropyLoss instead of something else, what BatchNorm and Dropout actually do, why the features need to be standardized, and so on.

What Surprised Me

The biggest surprise was how fast training was. The entire model trained in about two minutes on my laptop CPU. You hear about companies spending billions of dollars and months of compute time to train large language models, so I expected deep learning to be slow. But our model has roughly 46,000 parameters and processes 13 numbers per song - GPT-scale models have hundreds of billions of parameters and train on terabytes of text. The difference is something like six or seven orders of magnitude in scale, which puts the cost of frontier AI training into perspective.

I was also surprised that the model could predict genre at all from just 13 numeric features. Features like "danceability" and "valence" do not seem like the most obviously genre-defining characteristics, yet the model found enough signal to classify at 4.6x better than random chance. Classical and Metal were especially easy to identify - they have genuinely distinct audio signatures that show up clearly in the data.

What I Learned

- **PyTorch:** PyTorch is a deep learning framework built by Meta that lets you define neural networks and train them in Python. The key feature is automatic differentiation - when you call loss.backward(), PyTorch computes how to adjust every weight in the network to reduce the error, without you having to manually code the calculus. This is what makes training practical.
- **MLP architecture:** An MLP is the simplest type of neural network - just layers of neurons connected to each other. Each layer applies a linear transformation, then a non-linear activation function (ReLU). Stacking multiple layers lets the network learn complex, non-linear patterns that a single linear model like logistic regression cannot capture.
- **Problem framing matters:** Grouping 114 genres into 10 super-genres was a design decision that shaped the entire project. It taught me that how you frame a problem matters as much as what model you use. A messy catch-all category like "Dance/Other" actively hurts accuracy because it has no coherent audio signature.

Limitations and What I Would Try Differently

- **Genre overlap:** Genre boundaries are inherently fuzzy. A song labeled "indie" could reasonably be called "rock" or "pop" - even human listeners disagree. The confusion matrix shows this overlap clearly.

- **Limited input features:** The 13 numeric features miss lyrics, vocal style, instrumentation, and cultural context. Using spectrograms or raw audio with a convolutional neural network would likely perform much better.
- **Class imbalance:** Hip-Hop/R&B had only 4,000 samples vs. Latin/World's 20,000. Balancing the classes with oversampling techniques like SMOTE could improve recall for smaller genres.

References

- Spotify Tracks Dataset - https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset
- PyTorch Documentation - https://pytorch.org/docs/stable/
- scikit-learn Documentation - https://scikit-learn.org/stable/